AI Invoice & Expense Manager

Technical & Interview Preparation Guide

This guide is meant to teach the project well enough to defend it in an interview — not just describe what it does, but explain every design decision, every honest limitation, and every tradeoff. It intentionally matches the codebase: nothing here describes a technology or technique that isn't actually implemented in this repository.

Scope note: rather than a mechanically generated 220-question bank (20 questions x 11 categories), this guide has a curated ~65 questions across 10 categories, each at full depth. A shorter guide that gets actually studied beats a longer one that doesn't — the same principle the project itself applies to scope (see README, "Engineering quality over number of features").

Author: Ayush | Project: ai-invoice-expense-manager

1. Elevator Pitches

30-second explanation (memorize this)

It's an AI Invoice and Expense Manager — you upload a photo or PDF of a receipt, it runs OCR with Tesseract, pulls out the merchant, amount, date and a few other fields with regex-based extraction, normalizes the merchant name, and suggests a category. Nothing gets saved as real data until I review and confirm it on a verification screen — that gate is the one decision I'd defend hardest, because OCR is never perfect and a financial dashboard built on unverified numbers isn't trustworthy. Confirmed transactions feed a dashboard with deterministic analytics — monthly totals, category breakdowns, insight sentences — and there's an optional natural-language query layer that works with or without an LLM key, because the actual numbers always come from a database query, never from the model.

2-minute explanation

The problem I was solving is that tracking expenses from paper or PDF receipts is tedious enough that most people just don't do it, and most OCR-based tools I looked at treat extraction as a black box — you get a number with no way to tell if it's confident or a guess.

So the pipeline is: you upload a receipt, it goes through file validation, then Tesseract OCR with some OpenCV preprocessing — grayscale, upscaling small images, denoising, adaptive thresholding — because phone photos of receipts are noisy and unevenly lit. The raw OCR text then goes through a regex-based extraction layer that pulls out merchant, amount, date, currency, invoice number, tax, and payment method. For amount specifically, I prioritize a clearly labeled total over just grabbing the biggest number on the page, and I track whether the amount came from a real label or a fallback guess.

That extracted data never goes straight into the database as confirmed. It shows up on a verification screen where every field is editable, and only an explicit 'Confirm Transaction' click marks it confirmed — analytics only ever reads confirmed rows. That's the one architectural decision in the whole project that exists specifically because this is a financial tool, not a generic OCR demo.

For categorization, the default is a transparent rule-based lookup — known merchants map directly, otherwise keywords in the OCR text map to a category. I also built an ML classifier — TF-IDF plus logistic regression trained on a small synthetic dataset — but it's shown as a secondary suggestion, not the default, because I can defend exactly why a rule fired but a logistic regression's decision is a lot harder for a user to reason about, and the dataset is synthetic and small enough that I don't want to overstate what its accuracy numbers actually prove.

Analytics is plain pandas aggregation over confirmed transactions — monthly and weekly totals, category breakdowns, percentage changes, and a handful of insight sentences that are always built from an actual computed number, never phrased generically.

And then there's an optional natural-language query feature — you can type something like 'Food pe kitna kharcha hua?' and it'll answer with a real number. The intent parsing is

rule-based keyword matching, not an LLM call, specifically so the feature works with zero API keys. If you do configure an OpenAI key, it's only ever used to reword the answer I've already computed — the prompt literally tells it not to change the numbers, and if the call fails for any reason it just falls back to the template sentence. The whole point was that the LLM is additive, never load-bearing.

2. Full Architecture

The codebase is organized as one package per pipeline stage: app/ocr, app/extraction, app/normalization, app/classification, app/analytics, app/llm, app/database, app/models, app/utils, and app/ui. Each package only depends on the ones before it in the pipeline — analytics never imports from ocr, ocr never imports from the database. That's not a rule I imposed for its own sake; it falls out naturally from the fact that a receipt genuinely flows through these stages in one direction, and it's what makes each package independently testable (tests/ mirrors the same folder names). There's no ORM — app/database/repository.py is plain sqlite3 and hand-written SQL over one table. There's no REST API layer — the Streamlit pages call the app/ packages as normal Python function calls in the same process. Both of those are conscious decisions to not build architecture for problems this project doesn't have: one user, one process, one table that actually matters.

End-to-End Data Flow

```
receipt file -> app/utils/validators.py (type/size/corruption check) ->
app/ocr/pipeline.py (load the file, PDF or image) -> app/ocr/preprocess.py (grayscale,
upscale, denoise, adaptive threshold) -> app/ocr/engine.py (pytesseract, returns raw
text + mean word confidence) -> app/extraction/cleaning.py (light whitespace/line
cleanup) -> app/extraction/fields.py (regex extraction: merchant, amount, date,
currency, invoice number, tax, subtotal, payment method, plus a heuristic confidence
score) -> app/normalization/merchant.py (strip corporate suffixes, alias table,
fuzzy-match against known merchants) -> app/classification/categorize.py (rule-based
category + optional ML suggestion) -> inserted into SQLite with
confirmation_status='pending' -> shown on the verification screen
(app/ui/upload_page.py) -> user edits/confirms -> app/database/repository.py marks it
confirmed -> app/analytics/metrics.py and insights.py compute everything the dashboard
and the query engine show, reading only confirmed rows.
```

3. Concepts by Area

OCR

Optical Character Recognition (OCR) is the process of turning an image of text into machine-readable text. I use Tesseract, an open-source OCR engine, through the pytesseract wrapper. Before OCR runs, I preprocess the image with OpenCV: convert to grayscale (color carries no text signal), upscale if the image is smaller than 1000px on its longest side (Tesseract's LSTM models expect roughly scan-quality resolution), denoise with fastNlMeansDenoising (phone-camera sensor grain can look like extra character strokes), and apply adaptive thresholding to binarize the image using a locally-computed threshold rather than one global cutoff — that matters because a receipt photo often has uneven lighting, like a shadow across half the paper. I run Tesseract with --psm 6 ('assume a single uniform block of text') instead of the default --psm 3, because receipts are a dense narrow column of text, not a page with a detectable layout. Known OCR failure cases in this project: blurry/low-light photos, handwritten receipts (Tesseract's models are trained on printed text), non-standard layouts where labels don't match what my extraction regexes expect, and multi-page PDFs (I only process the first page).

NLP

The 'NLP' in this project is intentionally narrow and rule-based, not a trained language model, and I'd say that explicitly in an interview rather than oversell it. Two places use it: field extraction (regex patterns matching labels like 'Grand Total' or 'GST' in cleaned OCR text) and the natural-language query parser (keyword/regex matching for metric words, category words in English and Hinglish, and date-range phrases like 'last month' or 'pichle mahine'). There's no tokenization, embeddings, or a trained NLP model anywhere in this codebase — I chose keyword/regex matching deliberately because the actual vocabulary this app needs to understand (a bounded set of receipt labels, a bounded set of query phrasings) is small enough that a transparent, zero-dependency rule set solves it, and because it means the query feature works with no API key at all.

ML

The only trained model in this project is the category classifier: TF-IDF features (unigrams + bigrams) feeding a scikit-learn LogisticRegression. Feature extraction means converting text into numeric vectors — TF-IDF weights each word (or word pair) by how often it appears in a given document relative to how common it is across all documents, so distinctive words like 'biryani' or 'electricity' matter more than common ones. Classification is logistic regression predicting one of 12 category labels. Training uses a stratified 75/25 train_test_split with a fixed random_state so it's reproducible. Validation is the held-out 25% test set — I evaluate on data the model never saw during fitting, which is the whole point of a test split: training accuracy alone tells you almost nothing about generalization. I measure accuracy (fraction correct), macro precision and recall (averaged evenly across all 12 classes, so a class with fewer examples isn't drowned out), macro F1 (the harmonic mean of precision and recall), and a confusion matrix (rows = actual class, columns = predicted class, so you can see exactly

which categories get confused with which). The measured numbers are in docs/ML_METRICS.md — around 98% accuracy on this synthetic dataset, which I'm careful to explain reflects that the synthetic categories use fairly distinct vocabulary by construction, not that the model would perform that well on messy real-world receipts.

Analytics

Analytics here means pandas aggregation, not machine learning. app/analytics/metrics.py takes a DataFrame of confirmed transactions and computes: overview stats (total spend, count, average, largest expense via groupby-free operations like .sum()/.mean()/.idxmax()), category breakdown (.groupby('category')['amount'].agg(...) sorted descending, with each category's percentage share of the total), monthly and weekly spending (.dt.to_period('M') or ('W') then groupby), and percentage change between two periods (simple (current - previous) / previous * 100, returning None rather than infinity when the previous period is zero — an undefined percentage change should be represented as undefined, not as a number that looks real). app/analytics/insights.py turns those metrics into sentences, but every sentence embeds a real number from the metrics functions — there's no template that says 'spending is up' without the actual percentage attached, and insights that can't be computed (e.g. no data for the previous month) are simply not generated rather than guessed at.

Database

One table: transactions, in SQLite, accessed through plain sqlite3 (no ORM) via app/database/repository.py. The schema (app/database/db.py) has the fields you'd expect — merchant_raw and merchant (normalized) kept separately so I never lose the original OCR text, amount/currency/transaction_date/category as the core analytics fields, several genuinely-optional fields (invoice_number, tax, subtotal, payment_method) that aren't on every receipt, extraction_confidence as a heuristic score, and confirmation_status ('pending'/'confirmed') as the human-verification gate. There's one index: (confirmation_status, transaction_date), because nearly every query filters by status and sorts or filters by date — at this project's scale (a personal expense tracker, not a high-throughput system) that's the one index that actually pays for itself; I didn't add indexes speculatively. Queries are parameterized (using ? placeholders, never string-formatted SQL) specifically to avoid SQL injection, which matters even in a single-user local app because it's still the correct habit.

LLM

The LLM layer is opt-in and, by design, never computes a number. Every dashboard figure and every query answer comes from app/analytics running against SQLite before an LLM provider is ever consulted (see app/llm/query_engine.py). The intent parser (what category, what date range, what metric) is rule-based keyword matching, not an LLM call — partly so the feature works with zero API keys, and partly because the actual questions this app needs to understand are a bounded, known set. If an OpenAI key is configured, the LLM is handed the already-computed answer as a fact in the prompt ('do not alter the numbers') and asked only to reword it more naturally — that's the mechanism that prevents hallucination: the model is never in the loop for the arithmetic, only the phrasing. If the API call fails for any reason (bad

key, network issue, rate limit), the code catches the exception and falls back to the template sentence rather than surfacing an error to the user.

Security

Uploads are validated before anything else touches them — file extension allowlist, a 15MB size cap, an empty-file check, and a real corruption check (PIL's Image.verify() for images, a %PDF magic-byte check for PDFs). Uploaded files are never executed. .env (which would hold OPENAI_API_KEY) is gitignored; only .env.example with blank values is committed. data/app.db is gitignored — no real transaction data or receipts are ever committed to the repo, and the sample data included is entirely synthetic. SQL queries are parameterized against injection. The app has no authentication — that's a stated limitation (docs/LIMITATIONS.md), appropriate for a single-user local prototype but something I'd flag as the first thing to add before any real multi-user deployment.

Testing

53 pytest tests, one module per app/ package. They're mostly hand-calculated-value tests, not just 'does it run without crashing' — e.g. test_analytics.py builds a small fixed set of transactions, works out the correct totals by hand, and asserts the function returns exactly that. test_repository.py runs against a temporary SQLite file (never the real data/app.db) via a pytest fixture, so tests can't corrupt real data and don't depend on each other's state. Edge cases are tested deliberately: empty transaction lists, a previous-period value of zero for percentage change, missing/corrupted/oversized file uploads, and query-parser questions that don't specify a category or date range.

Deployment

The app runs locally with `streamlit run main.py`, requiring Tesseract and Poppler installed system-wide (pytesseract is a wrapper around the Tesseract binary, not an installation of it). docs/DEPLOYMENT.md documents a Streamlit Community Cloud path (with a packages.txt for the apt-level OCR dependencies) and a Dockerfile for self-hosting elsewhere. I did not deploy this to a public URL, and I say why directly: no authentication exists yet, so a public deployment would let anyone upload files to it with no login. I did verify the app runs correctly — all the screenshots in this repo are from a real running instance, including a real upload going through OCR end-to-end.

4. Code Walkthrough — Major Files

For each file: purpose, key functions, inputs/outputs, the important logic, the design decision behind it, real failure cases, and a question likely to come up about it.

app/ocr/pipeline.py

File	app/ocr/pipeline.py
Purpose	Orchestrates validation -> file loading -> preprocessing -> OCR into one function the UI calls.
Key functions	process_upload(filename, file_bytes) -> OcrResult
Inputs	Raw uploaded filename and bytes (from Streamlit's file_uploader).
Outputs	OcrResult(success, raw_text, ocr_confidence, error) — a dataclass, not an exception, so the UI can show a clean error message instead of a stack trace.
Logic	Validates the file first (app/utils/validators.py). Loads the first page as a PIL Image — pdf2image.convert_from_bytes for PDFs, PIL.Image.open for images. Runs preprocessing then OCR. Returns failure if OCR produces no text at all.
Design decision	Every failure path returns a result object with a specific, user-facing error string rather than letting an exception propagate — OCR on a real-world file has several distinct ways to fail (bad file, unreadable PDF, blank image) and each deserves a different message.
Failure cases	Corrupted PDF that pdf2image can't rasterize; a multi-page PDF where the content that matters is on page 2 (only page 1 is processed); a valid image that's entirely blank or unreadable text (returns success=False with a 'try a clearer photo' message).
Likely question	Why does this function return a result object instead of raising exceptions for OCR failures?

app/extraction/fields.py

File	app/extraction/fields.py
Purpose	Pull merchant, amount, date, currency, invoice number, tax, subtotal, and payment method out of cleaned OCR text using regex heuristics.
Key functions	extract_fields(raw_text) -> ExtractedFields; extract_amount, extract_date, extract_merchant, extract_currency, extract_payment_method as separately-testable helpers.

Inputs	Cleaned OCR text (a string, already whitespace-normalized by app/extraction/cleaning.py).
Outputs	ExtractedFields dataclass — every field is Optional; nothing is guessed into existence.
Logic	Amount extraction checks an ordered list of total-label regexes (grand total, total payable, amount paid, etc.) before falling back to 'largest number on the page' — and tracks which path was used (amount_source). Date extraction tries several format patterns and parses ISO year-first dates directly (dateutil's dayfirst flag mishandles them) versus dateutil with dayfirst=True for everything else, matching Indian date conventions. Merchant extraction takes the first line that isn't recognized boilerplate (TAX INVOICE, GSTIN, a URL, etc.).
Design decision	The AMOUNT_NUMBER regex uses negative lookaround so a number can't be picked up if it's glued to a letter or another digit — this exists specifically because an early version of this code misread a bill number like 'EB2026080099' as a ₹2 billion amount when no labeled total matched.
Failure cases	A receipt whose total uses a label phrasing not in TOTAL_LABELS (falls back to the largest-number guess, flagged via amount_source); a date format outside the four patterns handled; a merchant name that IS all boilerplate-looking text (returns None, and the verification screen shows it as blank for the user to fill in).
Likely question	Walk me through what happens when a receipt's total isn't labeled at all.

app/normalization/merchant.py

File	app/normalization/merchant.py
Purpose	Collapse merchant name variants ('SWIGGY', 'Swiggy Pvt Ltd', 'Swiggy Internet Pvt.') into one canonical name.
Key functions	normalize_merchant(raw_name, known_merchants=None) -> str
Inputs	The raw merchant string from extraction, and optionally a list of merchant names already in the database.
Outputs	A canonical, title-cased merchant name, or 'Unknown Merchant' if the input was empty.

Logic	Three layers in order: (1) strip a list of corporate-suffix regex patterns (Pvt Ltd, Ltd, Inc, Internet, .com, etc.); (2) look up the cleaned name in a small hand-maintained alias table for cases suffix-stripping doesn't fully resolve (e.g. Uber's printed legal name reduces to 'Uber India Systems', not 'Uber'); (3) if a list of known merchants is passed in, fuzzy-match against them (rapidfuzz token_sort_ratio, threshold 90) so a near-duplicate OCR misread collapses into an existing merchant instead of creating a lookalike new one.
Design decision	The alias table is deliberately short — a handful of real cases observed during development, not an attempt at a comprehensive merchant database. A giant hardcoded merchant list would be exactly the kind of fake-impressive complexity this project's brief explicitly said to avoid.
Failure cases	A merchant with an unusual suffix not in the corporate-suffix list; two genuinely different merchants with similar names that fuzzy-match above the threshold (a false merge) — the threshold of 90 is a manually chosen tradeoff, not tuned against a labeled dataset.
Likely question	Why maintain both a suffix-stripping step and a separate alias table instead of just one big lookup table?

app/classification/categorize.py

File	app/classification/categorize.py
Purpose	The single entry point the rest of the app calls to get a transaction's category — combines the rule-based baseline with an optional ML suggestion.
Key functions	categorize(merchant, raw_text, use_ml_suggestion=True) -> CategorizationResult
Inputs	Normalized merchant name and cleaned OCR text.
Outputs	CategorizationResult(category, method, ml_suggestion, ml_confidence) — category/method come from rules.py; ml_suggestion is shown alongside, never substituted in.
Logic	Calls categorize_by_rules() first (merchant table, then keyword table, then 'Other' default). If a trained classifier model file exists, loads it once (module-level cache) and calls predict_category() for a second opinion, returned separately.

Design decision	The ML suggestion is structurally incapable of silently overriding the rule-based category — the function signature returns both, and the UI (app/ui/upload_page.py) only shows the ML suggestion as a caption when it disagrees with the rule-based result. This was a deliberate choice to keep the more explainable system as the source of truth for a financial tool.
Failure cases	No trained model file present (predict falls back to ml_suggestion=None — doesn't error); a merchant/keyword combination that matches neither table (defaults to 'Other', method='default').
Likely question	If the ML classifier and the rule-based categorizer disagree, which one wins and why?

app/analytics/metrics.py

File	app/analytics/metrics.py
Purpose	Deterministic pandas aggregation over confirmed transactions — the single source of truth for every number the dashboard and the query engine show.
Key functions	overview_stats, category_breakdown, merchant_breakdown, monthly_spending, weekly_spending, spending_for_period, percentage_change
Inputs	A pandas DataFrame of transaction dicts, already filtered by the caller to confirmation_status='confirmed'.
Outputs	Plain dicts/lists (via .to_dict(orient='records')) so calling code (Streamlit, the query engine) never has to know pandas.
Logic	overview_stats uses .idxmax() to find the largest expense's full row, not just its amount. category_breakdown groups, sums, sorts descending, and computes each category's percentage share in the same pass. spending_for_period is the shared building block behind both the dashboard's 'This Month' figure and the natural-language query engine's date-scoped answers — written once, used both places.
Design decision	percentage_change returns None (not infinity or a huge number) when the previous period was zero, because an undefined percentage change genuinely is undefined — returning a large number would look like real data and mislead whoever reads it.
Failure cases	Empty DataFrame (every function has an explicit empty-input branch, tested in tests/test_analytics.py, returning zeros/empty lists rather than raising).

Likely question	Why is percentage_change allowed to return None, and what would happen downstream if it returned 0 or infinity instead?

app/llm/query_engine.py

File	app/llm/query_engine.py
Purpose	Executes a parsed question against the database and produces an answer — the piece that enforces 'the LLM never computes a number.'
Key functions	answer_query(question, transactions, provider=None, as_of=None) -> QueryAnswer; resolve_date_range(...)
Inputs	The raw question string, the list of confirmed transactions, and an LLMProvider (defaults to NullProvider).
Outputs	QueryAnswer(intent, result_value, template_answer, final_answer, used_llm) — result_value and template_answer are always populated; final_answer only differs from template_answer if an LLM successfully reworded it.
Logic	Calls parse_query() (rule-based intent parsing) then resolve_date_range() to turn a code like 'last_month' into concrete start/end dates, then dispatches on intent.metric to the right app/analytics call. Only after result_value and template_answer exist does it check provider.is_available and optionally call _reword_with_llm().
Design decision	The LLM call is wrapped in a try/except that falls back to template_answer on any failure — a network error or bad API key degrades the feature to 'plain sentence' rather than breaking it.
Failure cases	A question the parser can't confidently categorize (falls back to total_spending / this_month rather than erroring); an LLM provider configured with an invalid key (caught, falls back silently, used_llm=False).
Likely question	Show me exactly where in this file it would be possible for the LLM to change a number, if at all.

app/database/repository.py

File	app/database/repository.py
Purpose	All CRUD access to the transactions table — the only file in the app that contains SQL.

Key functions	insert_transaction, get_transaction, list_transactions (with optional filters), update_transaction, confirm_transaction, delete_transaction, get_distinct_merchants
Inputs	Transaction dataclass instances (insert) or plain field dicts (update/confirm) plus optional filter kwargs (list).
Outputs	Plain dicts (via sqlite3.Row -> dict), an inserted row's id, or None for a missing row.
Logic	list_transactions builds a WHERE clause dynamically from whichever filters are passed, using parameterized placeholders throughout — never string-formatted SQL. update_transaction restricts writes to an explicit allowlist of column names, so a caller can't accidentally (or maliciously) update a column like id or created_at.
Design decision	confirm_transaction is a thin wrapper around update_transaction that forces confirmation_status='confirmed' — it's the only path that can move a row from pending to confirmed, which keeps the human-verification gate enforceable at the data-access layer, not just in the UI.
Failure cases	Calling update_transaction with no allowed fields (returns without touching the database rather than running a no-op or malformed query).
Likely question	Why is there a column allowlist in update_transaction instead of just accepting whatever dict the caller passes?

5. Interview Questions & Answers

Organized by category. Each question has a short answer, a detailed answer, guidance on how to say it out loud, and a likely follow-up.

Project-Specific

Q1. Walk me through what happens end-to-end when a user uploads a receipt.

Short answer

Validate -> OCR -> extract fields -> normalize merchant -> suggest category -> show for verification -> confirm -> analytics.

Detailed answer

The file is validated for type/size/corruption, then loaded as an image (PDF pages via pdf2image). It goes through OpenCV preprocessing and Tesseract OCR to produce raw text. That text is cleaned and run through regex extraction for merchant, amount, date, currency, invoice number, tax, subtotal, and payment method. The merchant name is normalized, and a category is suggested by the rule-based categorizer (with an optional ML second opinion). All of this is inserted into SQLite as a 'pending' transaction. The verification screen shows every field, editable, and only an explicit 'Confirm Transaction' click marks it confirmed — which is the only state analytics reads.

How to say it out loud

I'd walk it stage by stage the way it's laid out in app/ — validation, OCR, extraction, normalization, categorization, then the verification gate before anything counts as real data.

Likely follow-up

What happens if OCR fails completely? — process_upload returns a result with success=False and a specific error message ('OCR could not read any text from this file') — nothing gets inserted into the database at all.

Q2. Why did you build the human verification step? Isn't that extra friction?

Short answer

Because OCR and regex extraction are both imperfect, and a financial dashboard built on unverified numbers isn't trustworthy.

Detailed answer

It is extra friction, and I decided that tradeoff is worth it specifically because this is a financial tool. If I let raw extraction go straight into 'confirmed' data, a single misread amount would silently corrupt the dashboard, and the user would have no way to know which numbers to trust. The verification screen also directly surfaces the extraction_confidence score and flags amounts that came from a fallback guess rather than a labeled total, so the user knows exactly when to scrutinize a field more carefully.

How to say it out loud

I'd frame it as a product decision I made deliberately, not something I'd cut if I were rebuilding this — it's the one thing in the architecture that exists specifically because it's a financial app.

Likely follow-up

Could you make verification optional for users who trust the extraction? — You could add a 'high-confidence auto-confirm' toggle for extractions above some confidence threshold, but I'd be cautious about it — it reintroduces the exact risk the gate exists to prevent, just for a subset of cases.

Q3. Why is the rule-based categorizer the default instead of the ML classifier?

Short answer

Transparency, no training-data dependency, and the ML model's honest evaluation doesn't support treating it as production-ready.

Detailed answer

A rule firing is traceable — you can look at rules.py and know exactly why a transaction got its category. The ML classifier's decision, while inspectable in principle, isn't something a user can reason about from the UI. The classifier is also trained on 240 synthetic, templated examples; its ~98% test accuracy reflects that the synthetic categories use distinct vocabulary by construction, not that it would generalize that well to messy real receipts. I didn't want to present that as production-ready, so it's shown as a secondary suggestion instead.

How to say it out loud

I'd be upfront that this was a judgment call about trustworthiness, and cite the actual measured accuracy and why I don't fully trust it yet.

Likely follow-up

What would make you switch the default to the ML model? — Enough real confirmed-transaction history to retrain it on actual data instead of synthetic templates, plus evaluating it against real receipts specifically, not just a held-out split of the synthetic set.

Q4. What was the hardest bug you ran into building this?

Short answer

A date parsing bug where dateutil's dayfirst=True flag mis-parsed unambiguous ISO dates, and a false 'amount' extraction where a bill number got picked up as a ₹2 billion charge.

Detailed answer

I tested the extraction pipeline against real synthetic receipts I generated, and found dateutil.parser.parse('2026-08-05', dayfirst=True) returns May 8th instead of August 5th — dayfirst apparently still reorders the trailing two fields even when the year is unambiguously first. I fixed it by parsing year-first ISO dates directly with datetime() instead of trusting dateutil's flag. Separately, on a synthetic electricity bill with bill number 'EB2026080099' and no cleanly labeled total match, the largest-number fallback picked up the digits inside that bill number as a ₹2,026,080,099 amount. I fixed it with a regex negative-lookaround so a number can only count as an amount if it isn't glued to a letter or another digit.

How to say it out loud

I'd tell this one concretely — it's a good story because it shows I actually ran the pipeline against real inputs rather than just trusting the code looked right.

Likely follow-up

How did you catch that bug? — I ran the full pipeline against the synthetic sample receipts and printed every extracted field for manual inspection — the ₹2 billion number was obviously wrong at a glance, which is exactly why I test against realistic inputs rather than only unit tests with hand-picked strings.

Q5. How would this system scale to thousands of users?

Short answer

It wouldn't, as built — SQLite is single-writer and there's no auth; I'd need Postgres, a real API layer, and background OCR processing.

Detailed answer

This is explicitly a single-user local prototype. SQLite handles one writer at a time, which is fine for one person clicking through a Streamlit app but not concurrent multi-user writes. I'd move to Postgres (the repository.py interface wouldn't need to change much, just the underlying SQL dialect), add authentication and per-user data isolation, and move OCR off the request path into a background job queue (OCR on a multi-page PDF can take a few seconds, which is fine synchronously for one user but wouldn't scale to concurrent uploads).

How to say it out loud

I'd answer this directly and not pretend the current architecture already scales — the honest answer is more credible than a vague 'it would scale fine.'

Likely follow-up

Why not just start with Postgres and a real API from the beginning? — Because the project's actual requirement was one user, learning the pipeline end-to-end — adding infrastructure for a scale problem I didn't have would have been exactly the kind of over-engineering the brief for this project explicitly warned against.

Q6. What would you do differently if you rebuilt this from scratch?

Short answer

Design the corrections-as-training-data loop from day one, and get real (even a handful of anonymized) receipts to test extraction against instead of only synthetic ones.

Detailed answer

Every confirmed edit already updates the category column in the database, but I didn't wire up retraining the classifier from that data — I'd design that loop in from the start next time. I'd also try to get even a small set of real, anonymized receipts (with permission) to validate extraction against, because synthetic receipts are cleaner and more predictable than real photos, and my confidence in the extraction heuristics is really only tested against my own synthetic generator's output.

How to say it out loud

This is a good one to answer honestly with a specific, technical improvement rather than something vague like 'I'd add more features.'

Likely follow-up

Isn't using synthetic data for both training and testing circular? — Yes, for the ML classifier specifically — that's exactly why docs/ML_METRICS.md is explicit that the high accuracy reflects the synthetic data's distinct vocabulary, not real-world generalization.

Q7. Why Streamlit instead of a React frontend with a REST API?

Short answer

One user, one process — Streamlit lets the whole app be Python, which matched the actual requirement without adding a second language and a network boundary.

Detailed answer

A REST API plus a separate frontend makes sense when you need a different team working on UI, multiple client types, or independent scaling of frontend and backend. None of that applied here. Streamlit let me build upload, verification, dashboard, and query pages that call the app/ packages as direct Python function calls — no serialization boundary, no separate deployment, and the whole codebase stays in one language, which also made it easier to keep the project at a scope I could fully understand and defend.

How to say it out loud

Frame it as a deliberate scope decision, and be ready to say what WOULD justify the heavier architecture.

Likely follow-up

What would push you toward a separate API layer? — A mobile app client, multiple frontend surfaces, or needing to scale the OCR/analytics backend independently of the UI — none of which this project's actual requirements included.

Q8. How does editing a transaction in the table affect the dashboard?

Short answer

Immediately — edits go through the same repository.update_transaction function analytics reads from, so there's no separate recalculation step.

Detailed answer

The transaction table (app/ui/transactions_page.py) uses Streamlit's data_editor, diffs the edited DataFrame against the original, and calls update_transaction() for any row with changed values. Because the dashboard and analytics always query the current state of the transactions table fresh on every page load, an edit is reflected the next time the dashboard re-renders — there's no cache or materialized view to invalidate.

How to say it out loud

Good place to mention there's no caching layer — simpler, and correct for this scale, but worth naming as a design choice.

Likely follow-up

Would that approach work if the dashboard had to handle a huge transaction history? — At some point you'd want to cache aggregates rather than recomputing pandas groupbys over the full table on every page load — not needed yet at this project's scale, but the first place I'd optimize if it became slow.

Q9. What's the confirmation_status field actually enforcing, mechanically?

Short answer

Every analytics/query function is called with status='confirmed' as an explicit filter — pending rows are structurally excluded, not just hidden in the UI.

Detailed answer

repository.list_transactions(status='confirmed') builds a SQL WHERE clause on confirmation_status. Every call site in app/ui/dashboard_page.py, transactions_page.py, and query_page.py explicitly passes status='confirmed'. So it's not that pending transactions are filtered out visually — they're never even fetched from the database for analytics purposes.

The only way a row moves from pending to confirmed is repository.confirm_transaction(), which is only called from the verification screen's Confirm button.

How to say it out loud

Point to the exact code path if asked to be specific — this is the kind of question where citing the actual function name shows you know the codebase, not just the concept.

Likely follow-up

Could a bug elsewhere in the code accidentally show pending data on the dashboard? — Only if a new call site forgot to pass status='confirmed' — there's no global filter enforced at the database layer, which is a real gap; a stricter design would default list_transactions() to confirmed-only unless explicitly overridden.

Q10. Why does insights.py sometimes generate fewer insights than expected?

Short answer

Because it only generates an insight when the underlying comparison is actually possible with real data — no data for last month means no month-over-month sentence.

Detailed answer

generate_insights() checks preconditions before generating each sentence — e.g. _month_over_month_insights returns an empty list if either the current or previous month's DataFrame is empty, rather than fabricating a comparison against zero. This was a deliberate choice: a fake 'infinite % increase' or a comparison against no data would look like a real insight but wouldn't be one.

How to say it out loud

This is a good example to have ready of 'graceful degradation over fabrication' as a principle running through the whole project.

Likely follow-up

What does the app show if there's no data at all yet? — generate_insights() returns a single explicit message: 'No confirmed transactions yet — insights will appear once you confirm some receipts.'

OCR

Q1. Why Tesseract instead of a cloud OCR API like AWS Textract or Google Document AI?

Short answer

Runs fully offline with no API key or billing, which matters for a project meant to be cloned and run by anyone evaluating it.

Detailed answer

Cloud OCR services generally outperform Tesseract on harder documents (rotated text, complex layouts, handwriting), and I say that directly rather than pretend Tesseract is state-of-the-art. But this project's inputs are printed receipts, which is within Tesseract's strengths, and requiring an interviewer or grader to sign up for a cloud account and pay for API calls just to run the project would undermine the whole point of a portfolio piece being easy to actually try.

How to say it out loud

Be honest that this is a tradeoff, not a claim that Tesseract is objectively best — that honesty is more convincing than overselling it.

Likely follow-up

How would you add a cloud OCR fallback? — app/ocr/engine.py's interface is run_ocr_with_confidence(image) -> (text, confidence) — a cloud provider implementation behind the same interface, selected by an environment variable similar to how app/llm/factory.py picks an LLM provider, would slot in without changing any other layer.

Q2. What preprocessing steps do you apply before OCR, and why each one?

Short answer

Grayscale, upscale if small, denoise, adaptive threshold — each addresses a specific real problem with phone-photo receipts.

Detailed answer

Grayscale removes color information that doesn't help text recognition. Upscaling (if the image is under 1000px on its longest side) compensates for messaging apps compressing photos below the resolution Tesseract's LSTM models expect. Denoising (cv2.fastNlMeansDenoising) removes sensor grain from low-light phone photos that can look like extra character strokes. Adaptive thresholding binarizes the image using a locally computed threshold per region rather than one global cutoff, which matters because receipt photos often have uneven lighting — a shadow across half the paper would blow out under a single global threshold.

How to say it out loud

Name all four and be ready to explain any one of them in more depth if asked — this is one of the questions most likely to get a 'why' follow-up on a specific step.

Likely follow-up

Why adaptive thresholding instead of a fixed global threshold? — A global threshold assumes uniform lighting across the whole image — a real receipt photo often doesn't have that, and adaptive thresholding computes the black/white cutoff locally so uneven lighting doesn't blow out half the image.

Q3. What does --psm 6 mean, and why did you choose it?

Short answer

Tesseract's page segmentation mode; 6 means 'assume a single uniform block of text,' which fits a receipt better than the default layout-detection mode.

Detailed answer

Tesseract's default (--psm 3) tries to detect a page layout — headers, columns, etc. — which is built for documents like scanned pages of a book or a form. A receipt is a single dense narrow column of text with no real layout to detect, and --psm 6 performed better on the sample receipts used during development because it doesn't waste effort trying to segment a layout that isn't there.

How to say it out loud

Good to mention this is a real, tested tradeoff — --psm 6 would perform worse on an actual multi-column layout, which I say explicitly in docs/OCR_PIPELINE.md.

Likely follow-up

What page segmentation mode would you use for a different kind of document, like a form? — Something closer to the default --psm 3, or a mode aware of sparse text / specific regions, since a form has real structure Tesseract needs to detect rather than assume.

Q4. What's the difference between the two confidence scores in this codebase?

Short answer

OCR confidence is Tesseract's own per-word character-reading confidence; extraction confidence is a hand-written heuristic about whether the right fields were found.

Detailed answer

run_ocr_with_confidence() in app/ocr/engine.py returns Tesseract's mean per-word confidence (0-100), which reflects how sure the OCR engine is about the characters it read — it says nothing about which word is the merchant vs. an address line. Separately, _estimate_confidence() in app/extraction/fields.py is a heuristic 0-1 score based on whether a labeled total was found (vs. guessed), and whether a date and merchant were found at all. It's not a calibrated statistical probability — it exists purely to decide when the verification screen shows an extra warning.

How to say it out loud

This distinction is worth being crisp about — conflating the two would be a real mistake to make out loud in an interview.

Likely follow-up

Could you combine them into one confidence score? — You could, but I kept them separate because they measure genuinely different things — character-level OCR certainty versus field-level extraction completeness — and collapsing them into one number would hide which kind of uncertainty you're looking at.

Q5. What are the known OCR failure cases in this project?

Short answer

Blurry/low-light photos, handwritten receipts, non-standard label phrasing, and multi-page PDFs beyond page one.

Detailed answer

Blurry or low-light photos produce noisy text that can defeat the labeled-total regex matching, forcing a fallback to the largest-number guess. Handwriting isn't something Tesseract's printed-text-trained LSTM models handle. A receipt using label phrasing not in my TOTAL_LABELS list (e.g. an unusual abbreviation) won't be recognized as a labeled total. And I only OCR the first page of a PDF — a total on page 2 of a multi-page document wouldn't be found.

How to say it out loud

Have these four ready as a list — this question comes up often and rewards a complete, specific answer over a vague one.

Likely follow-up

How would you detect that an extraction is unreliable rather than just wrong? — That's exactly what extraction_confidence and amount_source are for — a fallback ('largest_amount' instead of 'labeled_total') is a structural signal the value might be wrong, surfaced to the user rather than hidden.

Q6. Why do you only process the first page of a PDF?

Short answer

Receipts are almost always single-page; a multi-page PDF is more likely a statement, which this pipeline isn't built to segment.

Detailed answer

MAX_PDF_PAGES_PROCESSED = 1 in app/ocr/pipeline.py is an explicit, documented limitation, not an oversight. Supporting multi-page documents well would mean deciding which page(s) contain the actual transaction total, which is a different and harder problem than single-receipt extraction — I scoped it out rather than build a half-working version of it.

How to say it out loud

Frame this as a scoping decision you made consciously, with the reasoning ready, not as something you forgot to handle.

Likely follow-up

How would you extend it to handle multi-page documents? — OCR every page, then either let the user pick which page has the total, or extend the total-detection heuristic to search across all pages and take the most confident labeled match found anywhere in the document.

Q7. What's the difference between OCR and OCR post-processing in your pipeline?

Short answer

OCR (Tesseract) turns pixels into text; everything after that — cleaning, then regex extraction — is post-processing that turns text into structured data.

Detailed answer

app/ocr/ is strictly image-to-text: preprocessing an image and calling Tesseract. app/extraction/ is a separate concern — cleaning.py normalizes whitespace/line breaks in the raw OCR output, and fields.py runs regex heuristics over that cleaned text to pull out specific fields. Keeping them as separate packages means I can test extraction logic against hand-written text fixtures without needing an image or Tesseract at all (see tests/test_extraction.py) — the tests don't require Tesseract to be installed.

How to say it out loud

This is a good architecture question to connect back to testability, not just describe the split abstractly.

Likely follow-up

Could extraction ever run without OCR at all? — Yes — if the input were already digital text (a text-based PDF instead of a scanned image), you could skip OCR and feed extraction directly, though that path isn't built here since the product brief assumed photographed/scanned receipts.

Q8. How would you measure OCR accuracy properly, if you had to?

Short answer

Character error rate or word error rate against a labeled ground-truth transcription of real receipts — which I don't have, so I haven't claimed an accuracy number.

Detailed answer

The standard approach is comparing OCR output against a manually transcribed 'ground truth' for a labeled set of real documents, computing character error rate (CER) or word error rate (WER) via edit distance. I don't have a labeled real-receipt dataset for this project, so docs/OCR_PIPELINE.md and docs/LIMITATIONS.md are explicit that no formal OCR accuracy number is claimed — I only report Tesseract's own per-word confidence, which is a different thing entirely.

How to say it out loud

This is a chance to show you know the correct methodology even though you didn't have the data to run it — say both parts.

Likely follow-up

Why not just report Tesseract's confidence score as an accuracy number? — Because confidence and correctness aren't the same thing — Tesseract can be confidently wrong, especially on a clean-looking misread — so I keep them explicitly labeled as different things rather than implying one measures the other.

Q9. Does the OCR pipeline work on non-English receipts?

Short answer

Not currently — the code assumes English (Tesseract's default language model) and hasn't been tested with other languages.

Detailed answer

Tesseract supports multilingual OCR via language packs (e.g. --lang hin+eng for Hindi+English), but app/ocr/engine.py hardcodes the default English model, and none of the regex label matching in extraction (fields.py) accounts for non-English labels. This is a real,

stated future improvement, not something the current code handles.

How to say it out loud

Direct, honest answer — don't imply it works when it doesn't.

Likely follow-up

What would multilingual support require? — Installing the relevant Tesseract language packs, passing --lang to pytesseract, and extending the extraction label lists (TOTAL_LABELS, TAX_LABELS, etc.) with equivalents in each supported language.

ML & Classification

Q1. Why TF-IDF plus Logistic Regression instead of a neural network?

Short answer

With ~240 short synthetic examples, a neural network would overfit before learning anything generalizable, and would be harder to inspect than logistic regression's per-feature weights.

Detailed answer

Model complexity should match data size. A dataset this small can't support a model with enough capacity to meaningfully outperform a simpler one — a neural net would likely just memorize the 240 training examples' specific phrasing. Logistic regression over TF-IDF features is also directly inspectable: you can look at which words have the highest learned weight for each category, which matters when I'm being honest with myself (and an interviewer) about whether the model has learned something sensible or just latched onto a coincidental pattern.

How to say it out loud

This is a chance to show engineering judgment about matching model complexity to data size, not just knowing that logistic regression exists.

Likely follow-up

At what dataset size would a neural network become justified? — There's no fixed number, but you'd want at minimum several thousand real, diverse examples before the extra capacity of a neural network is likely to help rather than just overfit — and even then, I'd want a good baseline like this one to know whether the added complexity was actually earning its keep.

Q2. Explain precision, recall, and F1 in the context of your classifier.

Short answer

Precision: of transactions predicted 'Food,' how many actually were. Recall: of transactions that actually were 'Food,' how many got predicted correctly. F1 is their harmonic mean.

Detailed answer

For a specific category like Food: precision = true positives / (true positives + false positives) — how trustworthy a 'Food' prediction is. Recall = true positives / (true positives + false negatives) — how many actual Food transactions the model actually catches. I report macro-averaged precision/recall/F1 (each category weighted equally regardless of how many test examples it has), rather than micro-averaged, specifically so a category with fewer synthetic examples isn't drowned out in the average.

How to say it out loud

Have a one-sentence definition of each ready, and be ready to explain macro vs. micro averaging if pushed further.

Likely follow-up

Why macro-average instead of micro-average? — Micro-averaging would weight by class frequency, so with an even 20-per-category synthetic dataset it wouldn't actually differ much here — but macro-averaging is the more honest default in general because it doesn't let a dominant class hide poor performance on smaller ones.

Q3. What does the confusion matrix tell you that accuracy alone doesn't?

Short answer

Which specific categories get confused with which — accuracy is one number; the confusion matrix shows the error pattern.

Detailed answer

docs/ML_METRICS.md's confusion matrix shows, for example, exactly 1 Groceries example misclassified as Shopping in the test run (everything else was classified correctly). A single accuracy number wouldn't tell you that the model's one error was a reasonable near-miss (groceries and general shopping share some vocabulary) rather than something bizarre like confusing Food with Rent, which would suggest a real problem with the features.

How to say it out loud

Cite the actual measured result from this project (Groceries/Shopping) rather than speaking generically — it shows you looked at your own output.

Likely follow-up

Is a Groceries/Shopping confusion concerning? — Not especially — those categories can have genuinely overlapping vocabulary (e.g. 'Amazon' could plausibly appear in either context), so a small amount of confusion between adjacent categories is a reasonable, expected kind of error rather than a sign the model learned nothing.

Q4. Why is your classifier's accuracy around 98%, and should I trust that number?

Short answer

No — it's measured honestly but on synthetic, templated data with fairly distinct per-category vocabulary, so it overstates real-world performance.

Detailed answer

The 98% is a real, reproducible number from a genuine held-out test split (not training accuracy, not fabricated) — but the synthetic dataset was generated from templates where each category draws from a fairly distinct vocabulary pool by construction (Food templates always include words like 'Biryani'; Utilities templates always include 'electricity bill'). A classifier finds that easy to separate. It says the pipeline works end to end, not that it would perform this well on a messy real receipt for an unfamiliar merchant. I say this explicitly in docs/ML_METRICS.md rather than let the number speak for itself.

How to say it out loud

This is one of the most important questions to answer well — showing you understand WHY a good metric doesn't automatically mean a good model is exactly the kind of judgment this question is testing for.

Likely follow-up

How would you get a trustworthy accuracy number? — Evaluate against real, held-out receipts the model has never seen anything similar to during training — ideally collected independently of whatever process generated the training data, so there's no shared artificial pattern inflating the score.

Q5. What features does the classifier actually use?

Short answer

TF-IDF vectors (unigrams + bigrams) over the combined merchant name and OCR text.

Detailed answer

TfidfVectorizer(ngram_range=(1,2)) in app/classification/ml_classifier.py converts the input text into weighted word/word-pair frequency vectors — words that are distinctive to a document relative to the whole corpus get higher weight than words that appear everywhere. Bigrams (two-word sequences) capture some phrase-level signal a pure unigram model would miss, like 'electricity bill' as a unit rather than just 'electricity' and 'bill' separately.

How to say it out loud

Be ready to explain what a bigram adds over unigrams alone with a concrete example.

Likely follow-up

Why not use word embeddings instead of TF-IDF? — Embeddings could capture semantic similarity TF-IDF misses (e.g. 'cafe' and 'restaurant' being related even without shared words), but they'd add a real dependency and complexity that a dataset this small and this vocabulary-distinct doesn't clearly need — TF-IDF was the simpler tool that was actually sufficient here.

Q6. How did you split your data for training and evaluation?

Short answer

A stratified 75/25 train_test_split with a fixed random_state, so class proportions are preserved and results are reproducible.

Detailed answer

train_test_split(test_size=0.25, stratify=labels, random_state=42) in app/classification/ml_classifier.py. Stratifying ensures each of the 12 categories is represented in the same proportion in both the train and test sets — with only 20 examples per category, an unstratified random split could easily leave some category with very few or zero test examples, which would make its precision/recall meaningless.

How to say it out loud

Mention stratification specifically — it's the detail that shows you thought about the small-dataset problem, not just called train_test_split with defaults.

Likely follow-up

What would you do differently with a much larger dataset? — I'd move to a proper train/validation/test three-way split (or cross-validation) so I'm not tuning any choices — like the ngram range or regularization strength — against the same set I report final numbers on.

Q7. Is there a risk of overfitting with your classifier, and how would you detect it?

Short answer

Yes — I'd detect it by the test accuracy being much lower than training accuracy; here they're both very high because the dataset is small and templated.

Detailed answer

Overfitting shows up as a gap between training performance and held-out test performance. I only report test-set metrics precisely because training accuracy on a dataset this small would be close to 100% and would tell you nothing. The bigger risk here isn't classic overfitting within this dataset — it's that the whole dataset is a narrow, templated slice of what real receipts look like, so even a model that generalizes perfectly within this data distribution may not generalize to real-world text.

How to say it out loud

Distinguish overfitting-to-the-training-set from the deeper issue of the dataset itself not representing the real world — that distinction is the sophisticated part of this answer.

Likely follow-up

What regularization does LogisticRegression use by default in scikit-learn? — L2 (ridge) regularization by default, controlled by the C parameter — I used scikit-learn's default C rather than tuning it, since with a dataset this small and this easy to separate, tuning regularization strength wasn't the bottleneck.

Q8. How would you retrain the classifier on real usage data instead of synthetic data?

Short answer

Query confirmed transactions with their raw_text and category from SQLite, and feed that into the same train_and_evaluate() function instead of the synthetic CSV.

Detailed answer

Every confirmed transaction already has raw_text (the full OCR output) and category (possibly user-edited) stored in the database. scripts/train_classifier.py currently reads data/training_data.csv, but the same train_and_evaluate() function in app/classification/ml_classifier.py would work unchanged against a query like SELECT raw_text, category FROM transactions WHERE confirmation_status='confirmed' — the training code doesn't care where the (text, label) pairs came from.

How to say it out loud

This shows you designed the code to make this extension straightforward, not just that you know it's possible in the abstract.

Likely follow-up

How much real confirmed data would you want before doing this? — Enough to cover most of the 12 categories with a reasonable number of examples each — with too little, or categories missing entirely, stratified train/test splitting wouldn't even be possible.

Q9. Why didn't you use cross-validation?

Short answer

A single stratified train/test split was sufficient to get an honest, reportable number for a dataset and use case this size; cross-validation would be the next step for tuning hyperparameters rigorously.

Detailed answer

Cross-validation is most valuable when you're comparing multiple model configurations and want a more stable estimate of which one is actually better, or when your dataset is small

enough that a single split's results might be noisy. I did compare two models (logistic regression vs. Naive Bayes) informally during development, but for the final reported metrics, a single stratified split was enough to demonstrate the pipeline honestly without adding process complexity that wasn't needed for a proof-of-concept.

How to say it out loud

Fine to admit this as a scope decision, not a gap you're unaware of.

Likely follow-up

When would you add k-fold cross-validation to this project? — If I were tuning hyperparameters (like TF-IDF's ngram range, min_df, or logistic regression's C) systematically, or if the dataset were small enough that a single split's test accuracy might vary a lot depending on which examples happened to land in the test set.

NLP & Text Processing

Q1. Is what you built actually NLP, or just string matching?

Short answer

Mostly rule-based text processing — regex and keyword matching — not statistical or neural NLP, and I'd say that directly.

Detailed answer

Field extraction and the query intent parser are both pattern matching against known vocabulary, not tokenization/parsing/embeddings or any trained language model. The one place actual machine learning touches text is the category classifier's TF-IDF features, which is a standard NLP feature-engineering technique but a fairly basic one. I'm precise about this distinction rather than let 'NLP' sound more sophisticated than what's actually implemented.

How to say it out loud

This kind of honesty is exactly what this project's brief asked for, and it tends to land well in an interview — reviewers notice when a candidate doesn't oversell.

Likely follow-up

What would 'real' NLP add here that regex doesn't give you? — Robustness to phrasing you didn't anticipate — an embedding-based intent classifier could handle a novel way of asking 'how much did I spend' that doesn't match any of my regex patterns, at the cost of needing training data and being harder to debug when it's wrong.

Q2. How does your natural-language query parser handle Hindi/Hinglish?

Short answer

A small hardcoded keyword table (CATEGORY_KEYWORDS) mapping Hindi/Hinglish words like 'khana' or 'pichle mahine' to the same structured intent as their English equivalents.

Detailed answer

app/llm/query_parser.py's CATEGORY_KEYWORDS dict includes both English and transliterated Hindi words mapping to the same category ('food' and 'khana' both map to 'Food'), and separate regex pattern lists for date-range phrases include both ('last month' and 'pichl?e\s*mahine'). This is genuinely just a lookup table extension, not any kind of language detection or translation — it works because Hinglish speakers commonly mix English and transliterated Hindi words in the same sentence, which is exactly the pattern the product brief's example questions showed.

How to say it out loud

Be precise that this is vocabulary coverage, not language understanding — don't overclaim multilingual NLP capability.

Likely follow-up

Would this handle a question written entirely in Devanagari script? — No — the regex patterns match transliterated (Roman-script) Hindi words, not Devanagari text; that would need separate script-aware matching or transliteration first.

Q3. Why regex instead of a proper tokenizer/parser for field extraction?

Short answer

Receipt labels are a small, fairly standardized vocabulary — regex is sufficient, transparent, and needs no training data or extra dependency.

Detailed answer

A general NLP pipeline (tokenization, POS tagging, named entity recognition) is built for open-ended text where you don't know the vocabulary in advance. Receipt labels are the opposite — 'Total,' 'GST,' 'Invoice No' are a bounded, known set of phrasings. Regex matching against that known set is simpler to write, simpler to debug (a wrong match traces directly to which pattern fired), and doesn't need a trained model or extra dependency.

How to say it out loud

Tie this back to 'match the tool to the actual problem' as a recurring theme across the whole project.

Likely follow-up

What would break this approach? — A label phrasing genuinely outside my TOTAL_LABELS/TAX_LABELS/etc. lists — regex only recognizes what it's been explicitly given, which is exactly the tradeoff I document in docs/OCR_PIPELINE.md and docs/LIMITATIONS.md.

Q4. What text cleaning do you do before extraction, and why keep it minimal?

Short answer

Normalize line endings, collapse repeated whitespace/blank lines, strip empty lines — deliberately not aggressive, to avoid silently altering a digit or symbol.

Detailed answer

app/extraction/cleaning.py's clean_ocr_text() only touches whitespace and blank-line structure. I explicitly avoided anything like auto-correcting spelling or 'fixing' garbled characters, because in a financial document, an automated correction that guesses wrong about a digit or currency symbol is worse than leaving noisy text for the extraction regexes to work around (or fail cleanly on).

How to say it out loud

This is a good example of 'do less, deliberately' as an engineering decision worth explaining, not just describing the code.

Likely follow-up

Could aggressive cleaning ever help here? — For non-financial fields like the merchant name, maybe — but I chose to apply the same conservative cleaning uniformly rather than have different rules for different fields, to keep the pipeline simple and predictable.

Q5. How would you extend the query parser to handle a question outside its current keyword vocabulary?

Short answer

Add new keyword patterns to the existing lookup tables, or — for real open-endedness — swap the rule-based parser for an LLM-based one behind the same QueryIntent interface.

Detailed answer

The cheap extension is adding more entries to CATEGORY_KEYWORDS, MONTH_NAMES, or the metric/date-range pattern lists — that's how I'd handle a specific new phrasing I noticed users trying. For genuinely open-ended question understanding, I'd keep the QueryIntent dataclass as the contract and swap parse_query()'s implementation to call an LLM that outputs a QueryIntent-shaped JSON object — the rest of query_engine.py (which enforces that the LLM never computes the actual number) wouldn't need to change at all.

How to say it out loud

This shows the interface was designed with this exact extension in mind — a good sign of forward-thinking architecture.

Likely follow-up

Would an LLM-based parser risk reintroducing hallucination? — Not for the numeric answer, because query_engine.py's structure means the LLM would only ever be choosing which existing analytics function to call and with what parameters — it still wouldn't be the LLM doing arithmetic, just interpreting intent.

Q6. What's the risk of keyword-based category detection matching the wrong thing in a query?

Short answer

A false positive — e.g. a question that happens to contain a category keyword incidentally, not as its actual subject.

Detailed answer

_match_category() in query_parser.py does a straightforward regex search for any known category keyword anywhere in the question. A contrived question like 'Did I spend more on travel than on food?' would only match whichever keyword the regex list checks first (in dict iteration order), not correctly detect that the question is actually comparing two categories — that comparison intent isn't in my QueryIntent schema at all.

How to say it out loud

Good to have a concrete example of the limitation ready rather than just gesturing at 'it could go wrong.'

Likely follow-up

How would you support a comparison question like that? — Add a new metric type ('category_comparison') to QueryIntent with two category fields, detect a comparison pattern ('more than', 'vs') in the parser, and add a corresponding branch in query_engine.py's _execute_intent().

Q7. Does merchant normalization count as NLP?

Short answer

It's text normalization — regex suffix stripping, a lookup table, and fuzzy string matching — not NLP in the language-understanding sense.

Detailed answer

normalize_merchant() in app/normalization/merchant.py doesn't parse meaning; it's pattern-based cleanup (strip 'Pvt Ltd'/'Internet'/'.com' style suffixes), a small alias dictionary, and rapidfuzz's token_sort_ratio for approximate string matching against known merchants.

It's a standard data-cleaning technique used in a lot of real-world pipelines (deduplicating company names, addresses, etc.), and calling it 'NLP' would be overselling a fairly mechanical process.

How to say it out loud

Consistent with the project's overall stance: name things accurately rather than reaching for the more impressive-sounding term.

Likely follow-up

What's token_sort_ratio actually doing? — It tokenizes both strings, sorts each string's tokens alphabetically, then computes a similarity ratio between the sorted-token strings — which makes it robust to word order differences (e.g. 'Foods Swiggy' vs 'Swiggy Foods' would still score highly similar).

Database & Backend

Q1. Why no ORM (like SQLAlchemy)?

Short answer

One table, hand-written SQL is simple enough to read directly, and an ORM would add an abstraction layer this project's scale doesn't need.

Detailed answer

An ORM earns its keep when you have many related tables, complex joins, or want database-agnostic code. This project has one table. app/database/repository.py's raw SQL is short enough to read top to bottom and know exactly what query runs — an ORM would add a layer of translation between what I write and what SQL actually executes, which is friction without benefit at this scale.

How to say it out loud

Frame as 'right tool for the size of the problem,' the same theme as the no-REST-API and no-microservices decisions.

Likely follow-up

At what point would an ORM become worth it? — Multiple related tables with real foreign-key relationships and joins, or needing to support both SQLite and Postgres with the same code — neither applies here yet.

Q2. How do you prevent SQL injection?

Short answer

Every query uses parameterized placeholders (?) — user input is never string-formatted directly into SQL.

Detailed answer

Every conn.execute() call in app/database/repository.py passes values as a separate params tuple/list alongside a query string containing ? placeholders — sqlite3 handles safe substitution. list_transactions() builds its WHERE clause dynamically based on which filters are provided, but even there, only the column/operator structure is built as a string; the actual filter values always go through the parameterized params list, never interpolated into the SQL string itself.

How to say it out loud

Be ready to point to the exact pattern (? placeholders + params tuple) rather than give a generic 'I use parameterized queries' answer.

Likely follow-up

Is there anywhere in the codebase that's more at risk, like dynamic column names? — update_transaction() builds a dynamic SET clause from column names, but those names come from a hardcoded 'allowed' set intersected with the caller's dict keys — never from raw user input — so a column name can't be injected even though the SQL string itself is built dynamically.

Q3. Why is there only one index on the transactions table?

Short answer

(confirmation_status, transaction_date) — because nearly every real query filters by status and sorts/filters by date; I didn't add indexes speculatively.

Detailed answer

Every analytics and UI query starts from confirmation_status='confirmed' and often also filters or sorts by transaction_date, so that composite index directly serves the actual query pattern. I didn't index every column 'just in case' — extra indexes have a real cost (slower writes, more storage) that isn't worth paying without a query pattern that needs it, and at this project's data volume (dozens to low hundreds of rows) it barely matters either way, but the reasoning is the same one I'd apply at real scale.

How to say it out loud

Show you understand indexes have a cost, not just a benefit — that balance is the actual point of this question.

Likely follow-up

Would you add an index on merchant or category individually? — Only if I saw a specific slow query filtering heavily on just that column at real scale — I'd want to measure before adding, not guess.

Q4. How would you migrate this schema if you needed to add a new column?

Short answer

Manually, with an ALTER TABLE — there's no migration framework (like Alembic) wired up, which is a real gap for anything beyond a single-developer prototype.

Detailed answer

SCHEMA in app/database/db.py is a single CREATE TABLE IF NOT EXISTS statement, so it only handles the initial schema, not evolving it. Adding a column today would mean either deleting data/app.db and re-running init_db() (losing existing data — fine for this project's demo data, not fine for anything real) or manually running an ALTER TABLE against the existing file. A real migration tool would version schema changes and apply them incrementally without data loss — that's a genuine limitation I'd flag directly.

How to say it out loud

Don't dodge this one — naming the gap directly and knowing what tool would fix it (Alembic) is a better answer than pretending the current setup is fine for schema evolution.

Likely follow-up

Why didn't you add a migration tool from the start? — The schema didn't change enough during development to justify the overhead — but I'd add Alembic (or write manual migration scripts) before this went anywhere beyond a single-developer local project.

Q5. Explain the transaction schema's confirmation_status field and why it's a string, not a boolean.

Short answer

It's a string ('pending'/'confirmed') rather than a boolean specifically so it can grow into more states later (e.g. 'rejected', 'needs_review') without a schema change.

Detailed answer

A boolean confirmed/not-confirmed would work for exactly the two states this project currently has, but a string with defined valid values (tracked in app/models/transaction.py as PENDING/CONFIRMED constants) leaves room to add a third state later — like a 'rejected' status for a receipt the user decides was a duplicate or a mistake, rather than either confirming it or leaving it stuck as pending forever.

How to say it out loud

A small but real example of choosing a slightly more flexible representation for a good, specific reason — not flexibility for its own sake.

Likely follow-up

Doesn't a string field risk typos/invalid values a boolean wouldn't? — Yes — that's a real tradeoff. SQLite doesn't enforce an enum constraint the way Postgres could (a CHECK constraint or a native enum type), so an invalid status string could technically be written; the constants in transaction.py are a convention, not a database-level guarantee.

Q6. What happens to the raw_text field over time — doesn't it bloat the database?

Short answer

Yes, potentially — it's a real tradeoff between keeping the full OCR output for debugging/retraining versus storage growth; I chose to keep it.

Detailed answer

Every transaction stores its full OCR raw_text, which could be a few hundred to a couple thousand characters. For a personal expense tracker's scale (dozens to hundreds of transactions), this isn't a practical problem. I chose to keep it because it directly enables two things: the 'show raw OCR text' debug view on the verification screen, and re-training the classifier on real confirmed data later (see ML & Classification section) — both need the original text, not just the extracted fields.

How to say it out loud

Good example of naming a real tradeoff and explaining why you accepted it rather than pretending there's no downside.

Likely follow-up

How would you handle this at real scale, with millions of transactions? — I'd consider moving raw_text to separate cold storage (like S3) referenced by a key, rather than inline in the primary transactional table, so the hot table used for dashboard queries stays small and fast.

Q7. Why does list_transactions() return plain dicts instead of Transaction dataclass instances?

Short answer

Because it reads from the database (where every column, including nullable ones, is already resolved), and the callers (Streamlit, pandas) work more naturally with dicts than with a strict dataclass.

Detailed answer

insert_transaction() takes a Transaction dataclass as input, because that's a natural, type-checked shape to build in Python before writing. But list_transactions()/get_transaction()

return sqlite3.Row objects converted to plain dicts, because the direct consumers — pandas.DataFrame(transactions) in app/analytics/metrics.py, and Streamlit's data_editor — work naturally with dicts/records, and re-wrapping every row back into a dataclass just to immediately convert it again would be unnecessary ceremony.

How to say it out loud

This is a small but real 'why isn't this symmetric' question worth having a specific answer for rather than looking surprised by the inconsistency.

Likely follow-up

Doesn't that inconsistency (dataclass in, dict out) risk confusion? — A little — the tradeoff was optimizing for what each direction's actual consumers need rather than forcing a single representation everywhere; a stricter design might wrap read results in a dataclass too, at the cost of dataclass-to-dict conversions in analytics.

Q8. How do you handle concurrent writes to the database?

Short answer

SQLite's own file-level locking handles it correctly for this app's usage — a single Streamlit session, one write at a time — but it isn't built for real concurrent multi-user writes.

Detailed answer

Each repository function opens its own short-lived connection, does its work, and closes it (get_connection() -> ... -> conn.close()), rather than holding one long-lived connection — this minimizes how long any lock is held. SQLite handles a single writer correctly by default (it'll block/retry rather than corrupt data), which is sufficient for one person using the Streamlit app. It is not designed for many concurrent writers, which is exactly why docs/LIMITATIONS.md names this directly as a reason this wouldn't scale to multiple simultaneous users without moving to Postgres.

How to say it out loud

Name the actual mechanism (short-lived connections, SQLite's built-in locking) rather than a vague 'SQLite handles it.'

Likely follow-up

What would you change first to support multiple concurrent users? — Move to Postgres, which handles concurrent writers properly with MVCC, and add a connection pool instead of per-call connections.

LLM

Q1. How do you prevent the LLM from hallucinating a wrong number?

Short answer

By never asking it to produce one — the LLM only rewords an answer that's already been computed by app/analytics, and the prompt explicitly instructs it not to alter any numbers.

Detailed answer

In app/llm/query_engine.py, result_value and template_answer are both fully computed before provider.is_available is even checked. If an LLM is used, _reword_with_llm()'s prompt hands over the template answer as a stated fact ('Factual answer (do not alter the numbers)') and asks only for a rephrasing. There's no code path where the model's output becomes the source of a number shown to the user — the architecture makes hallucinated numbers structurally impossible, not just unlikely.

How to say it out loud

This is probably the single most important question in the whole interview guide — answer it precisely, pointing at the actual code structure, not just the policy.

Likely follow-up

Could the LLM still change a number by rewording carelessly, even with that instruction? — In principle yes, if it ignored the instruction — that's a real limitation of prompt-level constraints. A stricter version would validate that every digit in the LLM's output matches the template answer's digits before displaying it, and fall back to the template if they don't match; that validation isn't currently implemented.

Q2. Why is the intent parser rule-based instead of using the LLM to parse the question?

Short answer

So natural-language querying works with zero API keys, and because the actual question vocabulary this app needs to understand is small and known in advance.

Detailed answer

The product requirement was that core features — including natural-language querying — work without any LLM configured. A rule-based parser (keyword/regex matching, including Hindi/Hinglish keywords) is what makes that possible specifically for this feature. It's also a better-fit tool: the example questions in the brief are all answerable from a bounded set of metric words, category words, and date-range phrases, which is exactly what a keyword parser is good at.

How to say it out loud

Connect this back to the 'LLM is optional, never load-bearing' principle that runs through the whole project.

Likely follow-up

What's the ceiling of this rule-based approach? — It only understands the keywords it's been given — a genuinely novel phrasing outside that vocabulary falls back to default assumptions (total spending, this month) rather than failing, which is safer than guessing wrong but does mean there's a real limit to what it understands.

Q3. What happens if the OpenAI API call fails or times out?

Short answer

answer_query() catches the exception and falls back to the template answer — the user still gets a correct answer, just not LLM-reworded.

Detailed answer

The call to provider.complete() inside answer_query() is wrapped in a try/except that catches any exception (bad key, network failure, rate limit, timeout) and sets final_answer back to template_answer, with used_llm=False. This was a deliberate design choice: an LLM failure should degrade the feature gracefully to 'plain sentence' rather than break the query feature or show the user an error for something that isn't really broken — the correct answer was already computed regardless.

How to say it out loud

Point to the try/except specifically — this is a concrete, checkable claim about the code, not just a design philosophy.

Likely follow-up

Is that except clause too broad (catching all exceptions)? — It's intentionally broad because the point is 'anything that could go wrong with an external network call should degrade gracefully' — I'd only narrow it if I found a specific exception type I wanted to handle differently, which hasn't come up.

Q4. Why didn't you test the OpenAI provider against a real API key?

Short answer

No API key was available during development — I say that directly in the docs rather than imply it was tested when it wasn't.

Detailed answer

OpenAIProvider is implemented against OpenAI's documented chat completions API shape, and its is_available/complete() interface is exercised by tests up to the provider boundary (tests/test_query_engine.py tests the NullProvider path fully, since that's what's actually verifiable without a key). The live API call itself has not been run. docs/LLM_ARCHITECTURE.md and docs/LIMITATIONS.md both say this explicitly, because claiming something was tested when it wasn't is exactly the kind of dishonesty this project's brief said to avoid.

How to say it out loud

Don't soften this — state it plainly. Interviewers generally respect this kind of precision about what was and wasn't verified far more than a vague 'yes it works.'

Likely follow-up

How confident are you it would work if you added a key? — Reasonably confident, since it's a small, standard implementation against a well-documented API — but 'reasonably confident' and 'verified' are different things, and I'd want to actually run it before claiming it works.

Q5. How would you add a second LLM provider, like Anthropic's Claude?

Short answer

Write one new file implementing the LLMProvider protocol (complete(), is_available), and add one branch to the factory.

Detailed answer

app/llm/base.py defines LLMProvider as a small Protocol — anything with complete(prompt) -> str and an is_available property satisfies it structurally, no inheritance required. A new AnthropicProvider class implementing those two things, plus one new elif branch in app/llm/factory.py's get_default_provider() checking for an Anthropic-specific environment variable, is the entire change — nothing in query_engine.py or anywhere else would need to know a new provider exists.

How to say it out loud

This is a good example to have ready of the provider interface actually paying off, not just existing for its own sake.

Likely follow-up

Why a Protocol instead of an abstract base class? — A Protocol is structural typing — NullProvider and OpenAIProvider don't need to inherit from a common base class, they just need to have the right methods, which keeps each provider file simpler and avoids an unnecessary inheritance hierarchy for what's really just 'two functions.'

Q6. Does the LLM ever see raw transaction data?

Short answer

No — it only ever receives the user's question text and one already-computed answer sentence, never the underlying transaction rows.

Detailed answer

_reword_with_llm()'s prompt contains exactly two pieces of data: the user's original question and the template_answer sentence. It never receives the transaction list, merchant names, or any other raw data from the database. This matters for privacy as much as for hallucination prevention — README's Privacy Considerations section states this directly.

How to say it out loud

Connects the technical answer to the privacy angle — worth mentioning both if there's time.

Likely follow-up

Could this design still leak sensitive information indirectly? — The template answer itself could contain a specific merchant name or amount (e.g. 'You spent the most on X'), so if the API provider logs requests, that specific fact would be visible to them — which is a real, if narrow, privacy consideration for anyone using this with a live API key on genuinely sensitive data.

Q7. What's the difference between how you use the LLM here versus a typical 'chat with your data' AI product?

Short answer

A typical chatbot-over-data product often lets the LLM see raw data and generate both the answer and its phrasing; here the LLM never sees raw data and never generates the answer itself, only the phrasing of an answer already computed elsewhere.

Detailed answer

Many RAG-style 'ask your data' products feed retrieved rows or documents into the LLM's context and let it synthesize an answer directly — which is flexible but creates a real hallucination risk when the answer involves arithmetic the model has to do itself (LLMs are not reliable calculators). This project inverts that: deterministic code does 100% of the calculation, and the LLM's only job, when used at all, is making the already-correct sentence sound more natural. That's a narrower, safer use of the LLM specifically because the domain (financial figures) has zero tolerance for a wrong number.

How to say it out loud

This is a good chance to show you understand the broader industry pattern (RAG/chat-over-data) and can articulate specifically why you didn't use it here.

Likely follow-up

Are there parts of this app where a RAG-style approach WOULD make sense? — Possibly for something like 'summarize my spending habits in a paragraph' where there's no single correct number at stake — a more generative, LLM-driven answer would be reasonable there in a way it isn't for 'how much did I spend on Food.'

Q8. How do you keep API costs/usage bounded if this had real LLM traffic?

Short answer

The prompt is short (one question + one short answer, max_tokens=200), and the LLM is only ever called once per typed query, not per dashboard render.

Detailed answer

OpenAIProvider.complete() sets max_tokens=200 and temperature=0.2 — a short, deterministic-leaning completion, appropriate for 'reword this one sentence,' not an open-ended generation task. The LLM is only invoked from the Ask a Question page, on an explicit user action, never automatically on every dashboard load — the dashboard and analytics pages never touch the LLM layer at all, keeping any real-world API cost proportional to how often someone actually asks a typed question.

How to say it out loud

Good to have the specific numbers (max_tokens=200, temperature=0.2) ready rather than a vague 'it's efficient.'

Likely follow-up

Why temperature=0.2 instead of 0? — A low but non-zero temperature still allows some natural variation in phrasing (which is the whole point of rewording), while keeping the output close to deterministic — temperature=0 would be more rigid than necessary for a task that's explicitly about natural-sounding language, not precision.

System Design

Q1. How would you design this if it needed to support multiple users?

Short answer

Add a user_id column and auth, move to Postgres, and scope every query in repository.py by user_id — the interface shape wouldn't need to change much.

Detailed answer

The repository functions already take explicit parameters rather than reaching into global state, so adding a required user_id parameter to each of them (and a WHERE user_id = ? clause) is a relatively contained change. The bigger additions would be an actual authentication system (Streamlit itself doesn't have built-in multi-user auth — you'd need something like streamlit-authenticator or move the UI to a framework with session-based auth), and switching from SQLite to Postgres for real concurrent-writer support.

How to say it out loud

Have the concrete plan ready (user_id column, Postgres, auth library) rather than a vague 'add authentication.'

Likely follow-up

Would you keep Streamlit for a multi-user product? — For a real multi-user product I'd lean toward a proper frontend (React or similar) talking to a REST/GraphQL API — Streamlit's per-session single-process model isn't really built for that, which is part of why I scoped this project as single-user to begin with.

Q2. Where would OCR processing become a bottleneck, and how would you fix it?

Short answer

Synchronous OCR on the upload request blocks the UI for a few seconds; at any real scale I'd move it to a background job queue.

Detailed answer

Currently, process_upload() runs OCR synchronously inside the Streamlit request — fine for one user uploading occasionally, but it means the UI is blocked for however long Tesseract takes on that file. At scale, I'd push the OCR + extraction work onto a background task queue (Celery with Redis, or a cloud queue), return immediately with a 'processing' state, and update the transaction to 'pending verification' once the background job completes — the frontend would poll or use websockets to reflect the update.

How to say it out loud

Name a specific real tool (Celery/Redis) rather than a vague 'use a queue' — it signals you've actually thought through the mechanics.

Likely follow-up

How would the user experience change with async processing? — The upload would show a 'processing' status immediately instead of blocking, and the user would see the extracted fields appear on the verification screen once the background job finishes — a small UX change, but a necessary one for the backend change to make sense.

Q3. How would you cache dashboard analytics if recomputing them got slow?

Short answer

Cache aggregate results keyed by a hash of the relevant filter (e.g. month), invalidated whenever a transaction in that period is inserted/edited/deleted.

Detailed answer

Right now every dashboard load recomputes pandas aggregations from a fresh database query — fine at this project's data volume, but at real scale you'd want to avoid recomputing, say, all-time totals on every page view. A reasonable approach: cache monthly aggregates (Streamlit has @st.cache_data, or you'd use Redis for a non-Streamlit app) keyed by month, and invalidate the specific month's cache entry when a write touches a transaction dated in that month, rather than invalidating everything on every write.

How to say it out loud

Be ready to explain cache invalidation specifically — that's usually the harder part of this question, not caching itself.

Likely follow-up

What's the risk of caching here? — Showing stale numbers if invalidation is missed on some write path — which is exactly the kind of subtle bug that makes me cautious about adding caching before it's actually needed; premature caching is a common source of hard-to-find staleness bugs.

Q4. What single point of failure exists in this system, and how would you address it?

Short answer

The single SQLite file — if it's corrupted or the disk fails, all data is gone; I'd add regular backups and, at scale, move to a replicated database.

Detailed answer

data/app.db is a single file with no replication, no automated backups, and no encryption at rest — all explicitly named in docs/LIMITATIONS.md. For a real deployment, the immediate fix is scheduled backups (even a simple cron job copying the file to cloud storage); the more thorough fix is moving to a managed database service (like RDS/Cloud SQL Postgres) that handles replication and point-in-time recovery for you.

How to say it out loud

Name the actual current gap specifically (no backups, no replication) rather than answer this generically.

Likely follow-up

Would you add encryption at rest for a real deployment? — Yes — financial data warrants it, and a managed Postgres service typically offers this as a configuration option rather than something I'd have to build myself.

Q5. How would you version the API/data contract if the UI and backend were split into separate services?

Short answer

I'd version the QueryIntent/Transaction/analytics response shapes explicitly and treat app/ as the contract, adding fields rather than changing existing ones' meaning.

Detailed answer

Right now app/models/transaction.py's Transaction dataclass and app/llm/query_parser.py's QueryIntent dataclass are the de facto contracts between layers, even in-process. If split into services, I'd formalize those as an API schema (e.g. Pydantic models exposed via FastAPI, matching the project's sibling Resume Analyzer project's stack choice), version the API path (/v1/...), and follow additive-change discipline — add new optional fields rather than repurpose or remove existing ones, to avoid breaking whichever service hasn't been updated yet.

How to say it out loud

Fine to reference general API versioning practice here since this project doesn't have a real API layer to point to directly — just be clear you're speaking in general terms for this specific question.

Likely follow-up

Why didn't you build this as a versioned API from the start? — Because there was only ever one client (the Streamlit UI, in the same process) — versioning is a discipline that matters once multiple independent consumers exist, which wasn't this project's actual situation.

Q6. How would you monitor this system in production?

Short answer

Application logs for OCR/extraction failures and LLM fallback rate, plus basic uptime monitoring — none of which is built currently, since there's no production deployment.

Detailed answer

I'd want to log (not just silently handle) every OcrResult(success=False) and every LLM fallback-due-to-exception event, so I could see failure rates over time rather than only what an individual user sees in the UI. For a real deployment, I'd add structured logging (not print statements) and a basic uptime/error-rate dashboard — none of this exists in the current codebase, which is appropriate for a local single-user prototype but would be a first priority before any real production use.

How to say it out loud

Honest that this doesn't exist yet, paired with a concrete plan for what you'd add first.

Likely follow-up

What's the single most important metric you'd want visibility into? — The OCR/extraction failure rate — it's the leading indicator of whether the core pipeline is actually working for real users' receipts, versus everything downstream (categorization, analytics) which depends on extraction having worked in the first place.

Testing

Q1. What's your testing strategy for this project?

Short answer

53 pytest unit tests, one module per app/ package, focused on hand-calculated expected values and explicit edge cases rather than just 'does it run.'

Detailed answer

Each app/ package (extraction, normalization, classification, analytics, database, the query parser/engine, validators) has a corresponding tests/test_*.py module. Most tests construct a small, specific input, work out the correct output by hand, and assert exact equality — e.g. test_analytics.py's SAMPLE_TRANSACTIONS with hand-verified category totals. Edge cases (empty data, zero-division in percentage_change, corrupted/oversized uploads, unparseable dates) are tested explicitly, not left to chance.

How to say it out loud

Mention the specific number (53) and the hand-calculated-value pattern — it's more convincing than a vague description of 'good test coverage.'

Likely follow-up

What's NOT covered by these tests? — The Streamlit UI layer itself isn't tested (no UI/integration tests) — I verified the UI manually and with Playwright screenshots showing real, working output, but there's no automated test asserting the upload page's behavior the way there is for the underlying functions it calls.

Q2. Why do your database tests use a temporary file instead of an in-memory SQLite database?

Short answer

A temp file more closely matches real usage (connections opened/closed per call, like the actual repository functions do) and avoids any risk of touching the real data/app.db.

Detailed answer

tests/test_repository.py's temp_db fixture creates a real temporary file via tempfile.mkstemp(), sets DATABASE_PATH to point at it via monkeypatch, and removes it after the test. An in-memory database (':memory:') is faster, but each new sqlite3.connect(':memory:') call gets its own separate empty database — since repository.py's functions each open a fresh connection per call (rather than reusing one long-lived connection), an in-memory approach would mean every function call gets a blank database, which wouldn't work at all for this code's connection pattern.

How to say it out loud

This is a good 'why' to have ready — it's not an arbitrary choice, it's forced by how the connection-per-call pattern actually works.

Likely follow-up

Would you refactor the connection pattern to support in-memory testing? — I'd only do that if test speed became a real problem — 53 tests including the file-based database ones run in well under a second, so there's no actual performance issue motivating the change.

Q3. How do you test code that depends on 'today's date,' like the insights generator?

Short answer

generate_insights() and answer_query() both accept an explicit as_of parameter, defaulting to date.today() but overridable in tests.

Detailed answer

If these functions always used date.today() internally, tests would be flaky — passing today, failing tomorrow, or requiring awkward workarounds. Instead, generate_insights(df, as_of=None) and answer_query(question, transactions, as_of=None) both accept an explicit as_of date, defaulting to today() only when not provided. Tests pass a fixed date (e.g. date(2026, 8, 20)) so results are fully deterministic and reproducible regardless of when the test suite actually runs.

How to say it out loud

This is a specific, checkable design pattern (dependency injection of the 'current time') worth citing by name if asked about testability more generally.

Likely follow-up

Is this pattern used consistently everywhere the app needs 'now'? — Yes for the two places that compute date-relative analytics (insights and query answers); the database layer's created_at/updated_at timestamps use datetime.now() directly since those genuinely should reflect real wall-clock time when a row was written, not a testable input.

Q4. Give an example of a test that caught a real bug during development.

Short answer

test_strips_internet_pvt_ltd_suffix in test_normalization.py failed initially because standalone 'Pvt.' wasn't in the corporate-suffix list, only the combined 'Pvt Ltd' pattern was.

Detailed answer

I wrote a test expecting normalize_merchant('Swiggy Internet Pvt.') == 'Swiggy', ran the suite, and it failed — the actual output was 'Swiggy Pvt' because my CORPORATE_SUFFIXES list only had a combined r'pvt\.?\s*ltd\.?' pattern requiring both words together, and 'Internet' got stripped by a separate rule leaving 'Pvt.' standalone with nothing left to remove it. I added a standalone r'pvt\.?\b' pattern and the test passed. This is a real example, not a hypothetical — it's in this project's actual commit history.

How to say it out loud

Have this specific example ready — a concrete bug-and-fix story is much more convincing than an abstract claim that tests 'help catch bugs.'

Likely follow-up

How did you decide what test cases to write in the first place? — By thinking through actual variations I'd expect to see in real receipts (uppercase names, corporate suffixes, near-duplicate spellings) rather than only testing the happy path with one clean example.

Q5. How would you test the OCR pipeline itself, given it depends on an external binary (Tesseract)?

Short answer

I test extraction logic separately from OCR (with hand-written text fixtures, no Tesseract needed), and verified the full pipeline manually against synthetic generated receipt images.

Detailed answer

tests/test_extraction.py tests extract_amount/extract_date/extract_merchant/etc. against plain Python strings — no image, no Tesseract dependency, so these tests run anywhere Python runs. For the full OCR pipeline (image -> Tesseract -> extraction -> normalization -> categorization), I don't have automated tests, but I did run it end-to-end against scripts/generate_sample_receipts.py's synthetic images and manually inspected every extracted field — that's how I found and fixed the date-parsing and largest-number-fallback bugs.

How to say it out loud

Be clear about what IS automated (extraction logic) versus what was manually verified (the full OCR pipeline) — don't blur the two.

Likely follow-up

Why not automate the full-pipeline test too? — It's a reasonable next step — a test that runs process_upload() against a checked-in sample_receipts/ image and asserts specific extracted values would catch regressions in the full pipeline, not just the extraction-logic layer; I verified it manually this round but would want that automated for anything beyond a portfolio project.

Q6. What testing would you add before considering this production-ready?

Short answer

Automated full-pipeline tests against real (not just synthetic) receipts, load/concurrency testing, and integration tests for the Streamlit UI itself.

Detailed answer

Beyond what exists: (1) automated tests running the actual OCR pipeline against a checked-in set of sample images, not just extraction logic in isolation; (2) tests against real, messy receipt photos, not only clean synthetic ones — synthetic data can't reveal every real-world failure mode; (3) some form of UI/integration testing (Streamlit apps can be tested with tools like streamlit's own AppTest or Playwright-driven end-to-end tests, similar to how I used Playwright to verify the app manually and capture screenshots); (4) load testing if this were ever going to serve multiple concurrent users.

How to say it out loud

A good, specific answer for 'what's next' questions — shows you know the current test suite's actual boundary, not just that tests exist.

Likely follow-up

Which of these would you prioritize first? — Real (not synthetic) receipt testing — it's the one most likely to surface extraction failures that actually matter to a real user, versus concurrency/load testing which only matters once there's real multi-user traffic to handle.

General / Beginner

Q1. What does this project do, in one sentence?

Short answer

It turns a photo or PDF of a receipt into a categorized expense record, with a dashboard and a natural-language query feature on top.

Detailed answer

See the 30-second pitch at the start of this guide for the fuller version — but the one-sentence answer is deliberately about the outcome (categorized expense record, dashboard, queries) rather than the technology list, because that's what actually explains what the app does for a user.

How to say it out loud

Lead with the user outcome, not the tech stack — save 'Tesseract, scikit-learn, Streamlit' for when they ask how it's built.

Likely follow-up

Who would actually use this? — Students, freelancers, or anyone tracking personal/small-business expenses who currently either doesn't track receipts at all or does it entirely manually.

Q2. What's the difference between OCR and machine learning in this project?

Short answer

OCR (Tesseract) turns an image into text; machine learning (the optional classifier) is a separate, later step that predicts a category from that text.

Detailed answer

These are genuinely different technologies solving different problems. OCR is a mature, mostly-solved (for printed text) computer vision problem — recognizing character shapes in pixels. The category classifier is a text classification problem — given already-extracted text, predict which of 12 labels it belongs to. They're sequential in the pipeline (OCR has to happen before there's any text to classify) but are implemented with completely different techniques (Tesseract's LSTM models vs. scikit-learn's TF-IDF + logistic regression).

How to say it out loud

A good one to answer clearly and simply — this question is testing basic conceptual clarity, not depth.

Likely follow-up

Is OCR itself a form of machine learning? — Modern Tesseract (v4+) uses an LSTM neural network internally, so yes, in that sense OCR here does rely on a trained model — but it's a pretrained, general-purpose model I use as-is, not something I trained myself, which is an important distinction from the category classifier I did train.

Q3. What is a regular expression, and why did you use them so heavily?

Short answer

A pattern-matching language for finding structured text; I used them because receipt fields (labels, amounts, dates) follow fairly predictable text patterns.

Detailed answer

A regex describes a pattern to search for in text — e.g. r'\d{1,2}[/-]\d{1,2}[/-]\d{4}' matches a date like '26/08/2026'. Extraction, normalization, and query parsing in this project all lean on regex because receipt text and typed questions both have enough structure (known labels, known date formats, known keywords) that pattern matching is a simpler, more transparent tool than something heavier like a trained NER model.

How to say it out loud

Good to have one concrete regex example memorized (the date pattern is a clean one) to make this answer tangible rather than abstract.

Likely follow-up

What's a limitation of using regex for this? — It only recognizes patterns you've explicitly anticipated — a genuinely novel label phrasing or date format outside the patterns I wrote won't be recognized, which is a real, documented limitation (see docs/OCR_PIPELINE.md).

Q4. Why is a SQLite database used instead of just saving to a CSV file?

Short answer

SQLite supports proper querying (filtering, sorting), safe concurrent-enough access for one user, and atomic updates — a CSV would require re-writing the whole file for every edit.

Detailed answer

A CSV file has no query language — filtering transactions by date range and category would mean reading the whole file into memory and filtering in Python every time, and editing a single row means rewriting the entire file. SQLite gives indexed queries, atomic single-row updates, and a standard interface (SQL) that's easy to extend later. For a project with edit/delete/filter requirements like this one, a real database is the appropriate tool, not overkill.

How to say it out loud

Simple, direct comparison — good for showing basic database reasoning.

Likely follow-up

When would a CSV actually be the right choice? — For write-once, read-mostly data with no need for queries or concurrent edits — like the static training_data.csv used for classifier training, which is exactly why that one IS a CSV rather than a database table.

Q5. What is a dataclass, and why did you use one for Transaction?

Short answer

A Python class that auto-generates __init__, __repr__, and equality methods from a list of typed fields — used here so Transaction has one clear, typed definition shared across every layer.

Detailed answer

@dataclass on app/models/transaction.py's Transaction class means I just declare the fields (merchant: str, amount: Optional[float], etc.) and Python generates the boilerplate constructor and comparison methods. Using one shared dataclass instead of passing around loose dicts with the same keys everywhere means a typo in a field name is more likely to be caught (by

type checking / IDE support) rather than silently doing nothing at runtime.

How to say it out loud

Keep this simple and concrete — a good beginner-level question to answer clearly without overcomplicating it.

Likely follow-up

Why not use a plain class with __init__ written by hand instead? — It would work identically, just with more boilerplate code to write and maintain for the same result — dataclasses exist specifically to remove that repetitive code for simple data-holding classes like this one.

Q6. What is a confidence score, in plain terms?

Short answer

A number indicating how sure the system is about a value it produced — here, a heuristic 0-1 score about whether extraction likely got the right fields, not a guarantee of correctness.

Detailed answer

In this project specifically, extraction_confidence isn't derived from any statistical model — it's a hand-written heuristic that adds up points for whether a labeled total was found, whether a date was found, and whether a merchant was found. It's useful as a signal for when to show the user an extra warning, but it's explicitly not a calibrated probability (e.g. a 0.75 doesn't mean '75% chance this is correct' in any statistically rigorous sense).

How to say it out loud

Be precise about the difference between a heuristic score and a calibrated probability — that precision is worth demonstrating even on a 'beginner' question.

Likely follow-up

How would you make this a genuinely calibrated probability? — You'd need labeled ground-truth data (extractions marked correct/incorrect by a human) to actually fit a model predicting P(correct) from features like OCR confidence and which extraction path was used — a meaningfully bigger undertaking than the current heuristic.

Q7. What's the difference between a merchant and a category in this app?

Short answer

Merchant is who you paid (e.g. Swiggy); category is what kind of expense it was (e.g. Food) — one merchant usually maps to one category, but they're stored as separate fields.

Detailed answer

They're related but distinct pieces of information, and keeping them as separate database columns (rather than, say, only storing category and losing the specific merchant) matters because the dashboard's merchant-level insights (like 'Swiggy and Zomato account for 62% of Food spending') need the merchant, not just the category.

How to say it out loud

Simple distinction, good to answer crisply without over-explaining.

Likely follow-up

Could two different merchants ever have different categories on different transactions? — Not automatically in the current rule-based system — MERCHANT_CATEGORY maps a merchant

name to one fixed category — but a user can manually recategorize any individual transaction on the transaction table, which does allow per-transaction overrides.

Q8. Why does the app validate uploaded files before processing them?

Short answer

To reject obviously bad input early (wrong type, empty, corrupted, oversized) rather than let OCR fail confusingly on something that was never going to work.

Detailed answer

app/utils/validators.py checks file extension against an allowlist, rejects empty files, rejects files over 15MB, and does a real corruption check (PIL's Image.verify() for images, a %PDF magic-byte check for PDFs) — all before the file reaches OCR. This gives the user a clear, specific error message ('Unsupported file type' vs. 'File is empty' vs. 'Image file is corrupted') instead of a confusing downstream failure from Tesseract or pdf2image.

How to say it out loud

Frame this as good input-handling practice broadly, not just a receipt-specific detail.

Likely follow-up

Why check file extension instead of just trying to open the file and seeing if it works? — Extension checking is a fast, cheap first filter before doing any real work — but it's not sufficient alone (a file could have a .png extension and not actually be a valid PNG), which is exactly why there's also a real content-based corruption check afterward.

Tricky Follow-ups

Q1. If the rule-based categorizer and the ML classifier disagree, and the rule-based one is wrong, why not trust the ML model instead?

Short answer

Because I don't have evidence the ML model is more often right — its measured accuracy is on synthetic data, and I have no real-world comparison between the two.

Detailed answer

This isn't a case where I'm confident the rule-based system is always right — merchant/keyword tables can definitely miss or misclassify. The honest answer is I chose the more explainable default because I couldn't defend the ML model's real-world accuracy with the evaluation I actually have (synthetic data). If I had a labeled set of real disagreements between the two systems, with ground truth, I could make an evidence-based call about which to trust more — I don't have that yet.

How to say it out loud

This question is testing whether you'll defend a position past the evidence — the right move is to say plainly what you don't know.

Likely follow-up

How would you gather that evidence? — Log every case where the rule-based and ML categorizations disagree, and use the user's final confirmed category (after they review it) as ground truth — over time that builds exactly the real-world comparison dataset needed.

Q2. Your ML classifier gets 98% accuracy but you say not to trust it — isn't that contradictory?

Short answer

No — a metric can be a real, correctly-measured number and still not generalize to a different, harder distribution of data than what it was measured on.

Detailed answer

98% accuracy on the held-out synthetic test set is a true, reproducible fact about how the model performs on data from the same distribution as its training data. My claim isn't that the number is wrong — it's that the training/test distribution (templated synthetic text with distinct per-category vocabulary) is easier than real, messy OCR'd receipts, so the same model would likely score lower on the harder, real distribution. That's not a contradiction, it's the standard distinction between in-distribution and out-of-distribution performance.

How to say it out loud

This is exactly the kind of pushback question worth practicing out loud — staying calm and precise about the distinction matters more than the content itself.

Likely follow-up

How would you actually quantify the gap between synthetic and real-world performance? — Collect and label a set of real receipts, run the model against them, and directly compare that accuracy to the 98% synthetic-test number — until that's done, the size of the gap is genuinely unknown, not just assumed to be large.

Q3. Couldn't you have just used GPT-4 to extract every field directly from the receipt image, instead of building all this OCR/regex machinery?

Short answer

You could, and it might even work better on hard cases — but it would cost money per receipt, require an API key to run the core feature at all, and be harder to debug when it's wrong.

Detailed answer

A vision-capable LLM extracting fields directly is a real, increasingly common approach, and I wouldn't claim my regex pipeline beats it on raw accuracy for hard receipts. But it would violate this project's actual requirement that core features work with zero API keys, would add a per-receipt cost that a free/local pipeline doesn't have, and — importantly for a portfolio project — would replace something I built and can fully explain with an API call whose internal reasoning I can't inspect or defend in the same depth.

How to say it out loud

Don't be defensive about this one — acknowledge the alternative is legitimate, then give your specific, real reasons for not choosing it.

Likely follow-up

Would you ever add that as an option? — Yes, as an optional alternative extraction path behind the same interface OCR uses now — similar to how the LLM query layer is optional — for users who have an API key and want potentially better accuracy on hard receipts, while keeping the free/local path as the default that always works.

Q4. You said the LLM never computes numbers — but doesn't the query PARSER deciding which category/date-range to filter by also involve a kind of 'computation' that could be wrong?

Short answer

Yes — intent parsing can genuinely misinterpret a question, and that's a real error mode distinct from, but related to, hallucination.

Detailed answer

This is a fair distinction to draw. 'The LLM never computes the number' protects against a specific failure mode — a fabricated or arithmetically wrong figure. It does NOT protect against the parser (rule-based here, not even LLM-based) choosing the wrong intent — e.g. misreading which category or date range the user meant. If that happens, the app confidently returns a real, correctly-computed number... for the wrong question. That's a genuine limitation, and the 'How this was answered' expander on the Ask a Question page exists specifically so a user can catch that kind of misinterpretation, since the number itself will always look legitimate.

How to say it out loud

This is a genuinely sharp follow-up — acknowledging the distinction clearly, rather than conflating 'no hallucinated numbers' with 'no possible wrong answers,' is exactly the right move.

Likely follow-up

How would you reduce intent-misparsing errors? — Show the parsed intent to the user before executing the query (rather than only after, in a collapsed expander) so a misparse is visible

immediately, or add a confirmation step for low-confidence parses — similar in spirit to the extraction confidence gate already used for receipts.

Q5. Your merchant fuzzy-matching threshold is 90 — how do you know that's the right number, versus just something that seemed reasonable?

Short answer

I don't have rigorous evidence it's optimal — it's a manually chosen, documented tradeoff, not a tuned value, and I say that directly in docs/LIMITATIONS.md.

Detailed answer

FUZZY_MATCH_THRESHOLD = 90 in app/normalization/merchant.py was chosen by trying it against a handful of test cases (genuine near-duplicates like 'Swigy' vs 'Swiggy' matching, clearly different merchants not matching) and picking a value that handled those correctly. That's a reasonable starting point, not a rigorously tuned parameter — properly tuning it would need a labeled dataset of merchant-name pairs marked 'should match' / 'should not match,' computing precision/recall at different thresholds, and picking the threshold that best balances false merges against missed matches.

How to say it out loud

Admitting 'I picked a value that worked on my test cases, not a tuned optimum' is a stronger answer than pretending 90 is scientifically derived.

Likely follow-up

What would go wrong with a threshold that's too low, versus too high? — Too low risks false merges — two genuinely different merchants (like 'Star Bazaar' and 'Star Bucks') getting collapsed into one; too high risks missing real near-duplicates that should have merged, leaving fragmented merchant data on the dashboard.

Q6. If a user uploads someone else's receipt or a fraudulent one, does your system catch that?

Short answer

No — this system has no fraud detection at all; it trusts that whatever the user uploads and confirms is a legitimate representation of their own spending.

Detailed answer

There's no verification that a receipt is genuine, unaltered, or belongs to the uploading user — the system extracts and stores whatever data is on the image. This is appropriate for the stated use case (a personal expense tracker where the user has every incentive to enter their own real data accurately) but would be a serious gap for any use case involving reimbursement, audit, or multi-party trust, where someone might have an incentive to submit altered or fraudulent receipts.

How to say it out loud

Good to state this limitation plainly and specifically rather than being caught off guard — it's an obvious gap once named, so naming it yourself is stronger than being asked and hedging.

Likely follow-up

What would fraud detection even look like for this kind of system? — Image forensics (detecting signs of digital editing), cross-referencing extracted merchant/amount against

known merchant patterns for anomalies, and — most practically for a reimbursement context — requiring the original file retained and auditable rather than just the extracted fields, none of which this project implements.

Q7. You built both a rule-based AND an ML categorizer — isn't that redundant engineering effort for a portfolio project?

Short answer

I'd defend it as intentional, not redundant — building both let me make and document an evidence-based comparison, which is a more honest and more impressive story than only ever having one approach.

Detailed answer

If I'd built only the rule-based system, I couldn't speak with any authority about whether ML would help here, or explain what a real precision/recall/F1 evaluation looks like for this problem. If I'd built only the ML system, I couldn't offer a transparent, zero-training-data default, and I'd be less able to explain WHY the classifier's accuracy number doesn't mean what it might look like it means. Building both, and being explicit about why one is the default, is the more complete and more honest engineering story — not redundancy for its own sake.

How to say it out loud

This is a good moment to connect back to the project's stated priority — 'engineering quality over number of features' — and explain why this specific case doesn't violate that.

Likely follow-up

Was there anything you built that, in hindsight, WAS redundant or unnecessary? — I don't think so within app/ itself, but I was deliberate about NOT building several things the original brief mentioned as optional — a corrections table separate from the transactions table, for instance — because the existing schema already supported the same need without adding a new table.

Q8. How do you know your synthetic sample receipts are representative enough to have caught the real bugs you found?

Short answer

I don't know that they're fully representative — I know they caught two real, non-trivial bugs, which is evidence they're useful, not evidence they're sufficient.

Detailed answer

The date-parsing bug and the largest-number-fallback bug were both genuinely caught by running the pipeline against synthetic receipts I generated, which is real evidence this testing approach has value. But synthetic receipts are, by construction, cleaner and more predictable than real photographed receipts — different lighting, angles, creases, and truly unpredictable layouts that a real photo would have. I'd treat 'my synthetic tests caught two real bugs' as encouraging, not as proof the extraction pipeline is robust to everything a real user would throw at it.

How to say it out loud

This is testing whether you'll overstate what your own evidence proves — resist the pull to claim more confidence than you've actually earned.

Likely follow-up

What's the fastest way to get real-world validation without a large data-collection effort? — Ask a handful of people (with consent) to try uploading their own real receipts and report what broke — even five or ten real photos from different phones/lighting conditions would surface failure modes synthetic data can't, at very low cost.

6. CV / Resume Bullet Points

Three versions for different audiences. All are truthful to what was actually built and measured — no invented users, accuracy figures, or business impact.

Technical version

- Built an end-to-end OCR-to-analytics pipeline (Python, Tesseract, OpenCV) that extracts structured fields from receipt images/PDFs using regex-based heuristics with confidence scoring and graceful handling of missing data.
- Designed a SQLite schema and a parameterized, ORM-free data access layer with 53 passing pytest tests covering extraction, normalization, categorization, analytics, and edge cases (empty data, corrupted uploads, malformed input).
- Implemented a two-tier expense categorization system: a transparent rule-based baseline plus an optional scikit-learn (TF-IDF + Logistic Regression) classifier, evaluated with accuracy/precision/recall/F1 and a confusion matrix on a held-out test split.
- Built a deterministic pandas analytics engine (monthly/weekly aggregation, category breakdowns, percentage-change calculations) that never depends on an external service for its numeric output.

AI/ML version

- Designed an OCR + regex-based document-extraction pipeline for financial receipts, with explicit confidence scoring and a human-in-the-loop verification step to guard against silent extraction errors reaching downstream analytics.
- Trained and honestly evaluated a TF-IDF + Logistic Regression text classifier for expense categorization on a documented synthetic dataset, with the tradeoffs between the rule-based baseline and the ML model explicitly analyzed and written up.
- Architected an optional LLM layer (provider-swappable interface, OpenAI implementation) constrained so the model only rewords already-computed answers and never performs the underlying calculation — designed specifically to prevent numeric hallucination in a financial context.
- Built a rule-based natural-language query parser (English/Hinglish keyword and date-range matching) that maps free-text questions to structured database queries without requiring any LLM API access.

Product version

- Built an AI-assisted expense management web app (Streamlit) that turns a photo of a receipt into a categorized transaction and a live spending dashboard — upload, OCR extraction, human verification, and analytics in one flow.

- Designed a human-verification UX step so users review and can correct every OCR-extracted field before it counts toward their spending dashboard, balancing automation with the accuracy financial data requires.
- Shipped a spending-insights dashboard (category breakdown, monthly trend, month-over-month change) with data-backed insight sentences and a natural-language question-answering feature, fully functional with or without an AI API key configured.

7. Concepts to Revise Before an Interview

- Why the confirmation gate exists and exactly which function enforces it (repository.confirm_transaction).
- The amount extraction priority order (labeled total > fallback largest number) and why the fallback is flagged.
- The date-parsing bug (dateutil dayfirst on ISO dates) — a concrete story of a real bug found and fixed.
- Why rule-based categorization is the default, not the ML classifier — and what would change that.
- What TF-IDF, precision/recall/F1, and a confusion matrix each mean, using this project's actual numbers.
- The exact mechanism preventing LLM hallucination (template answer computed before any LLM call; prompt says 'do not alter numbers'; exception fallback).
- Why the query intent parser is rule-based, not LLM-based, and its real vocabulary limits.
- The database schema, the one composite index, and why no ORM.
- What is NOT tested (live OpenAI key, full-pipeline automation, UI integration) — stated honestly, not glossed over.
- The three CV bullet versions — know which one fits which conversation.